AI 시스템반도체 SW개발자(2기) 프로젝트 결과보고서

# PARKING TOWER

주차 타워 입출차 관리 시스템



3팀 윤홍은 조성수 신유지 김형진

# 프로젝트 주제

STM32 M4 ↔ QT 연동 프로젝트

## 프로젝트 주제

STM32와 QT를 연동한
**주차 관리 시스템**

- ✓ **차량 감지**
- ✓ **주차 공간 관리**
- ✓ **주차 공간 배정**
- ✓ **차단기 제어**
- ✓ **입출차 관리**

## 개발 목표

**STM32 → 센서 이벤트 감지 · 주차 공간 배정 | QT → 차량 · 요금 관리**

**1. 차량 감지**
- 초음파 센서를 이용한 차량 접근 감지
- 차량 접근 시 QT에 이벤트 전달

**2. 주차 공간 관리**
- 1~8층 총 8개 주차 공간
- 층별 LED를 이용한 점유 상태 표시
- 만차 상태 판단

**3. 주차 공간 배정**
- 빈 공간 중 가장 낮은 층 자동 배정
- 배정 결과를 QT에 전달

**4. 차단기 제어**
- 입차/출차 시 서보 모터 제어
- 일정 시간이 지나면 자동 닫힘

**5. 입출차 관리**
- 차량번호 관리
- 주차 시간 계산
- 요금 계산 및 정산

# 동작 시나리오

입차 → 주차 → 출차 → 정산

**① 차량 감지**

초음파 센서로
접근 차량 감지

→

**② 입차**

입차 버튼 활성화,
QT GUI에
차량번호 입력

→

**③ 주차**

차단기 열림
주차 완료

- Ready 수신 → 슬롯 상태 요청
- 주차 완료 → 차량 등록 → 3초 후 상태 갱신

↓

**④ 출차 검색**

차량번호 뒤 4자리로
차량 검색

→

**⑤ 요금 정산**

주차 시간 계산
요금 표시

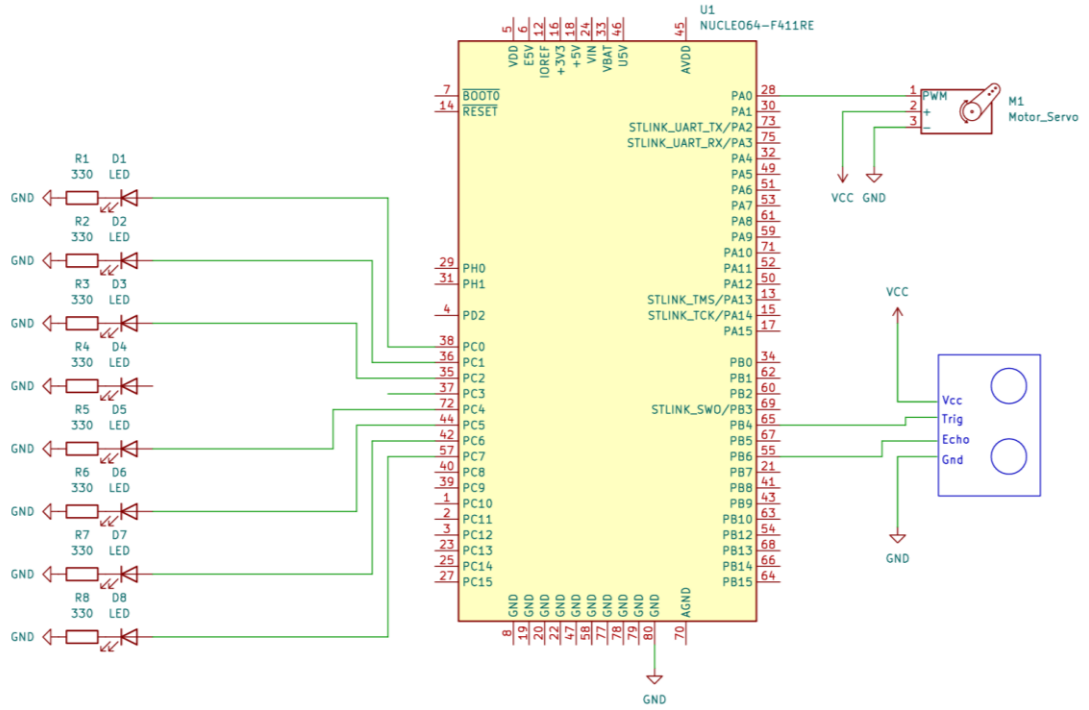
→

**⑥ 출차**

Qt → O:n
STM32 제어

- 출차 완료 → 차량 제거 → 완료 화면

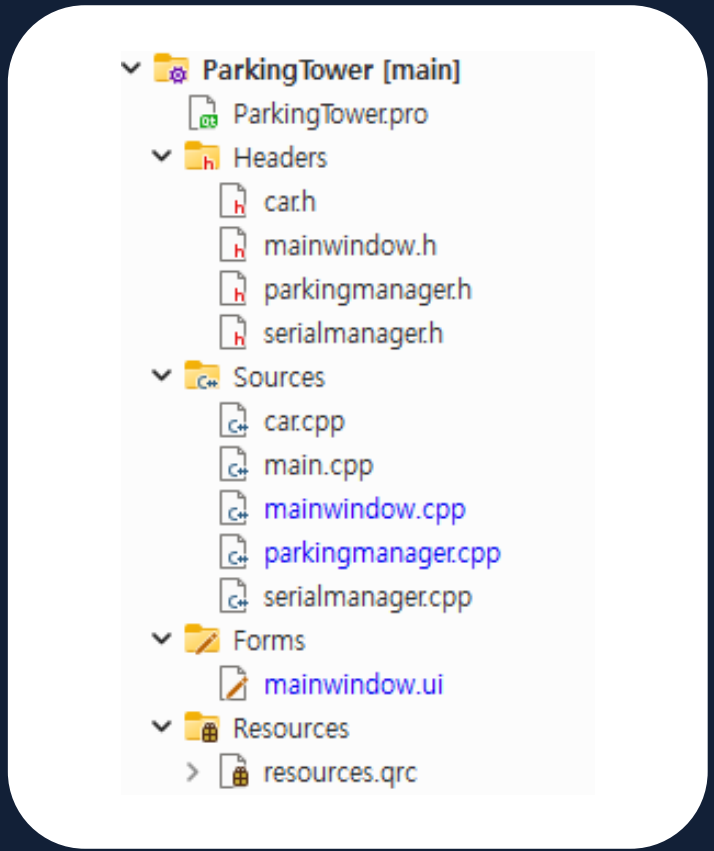
# 회로도 / 핀맵 / 파트리스트

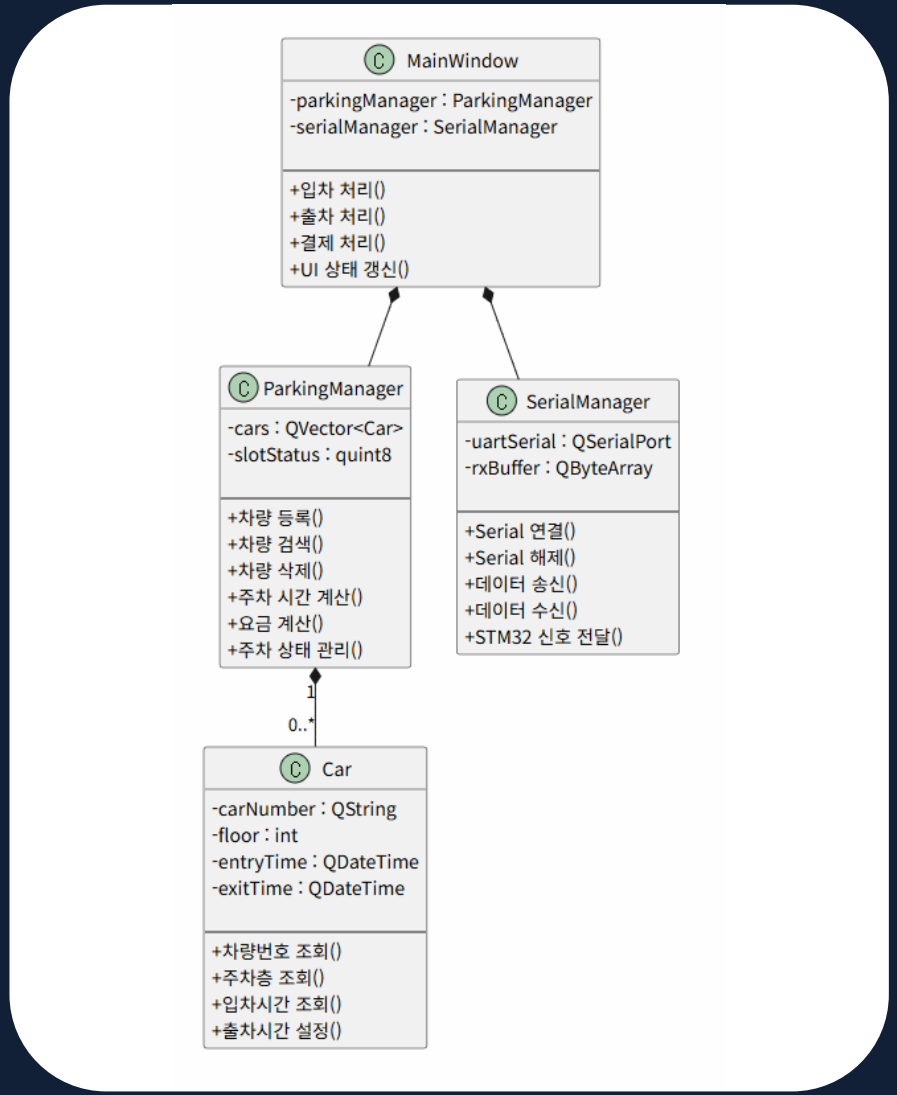



- 서보 모터: PA0
- 초음파 센서 TRIG: PB4
- 초음파 센서 ECHO: PB6
- LED: PC0 ~ PC7

| No. | 부품명 | 부품번호 | 개수 |
|---|---|---|---|
| 1 | STM32 M4 | NUCLEO-F411RE | 1 |
| 2 | 서보 모터 | SG90 | 1 |
| 3 | 초음파 센서 | HC-SR04 | 1 |
| 4 | LED | | 8 |
| 5 | 330Ω 저항 | | 8 |

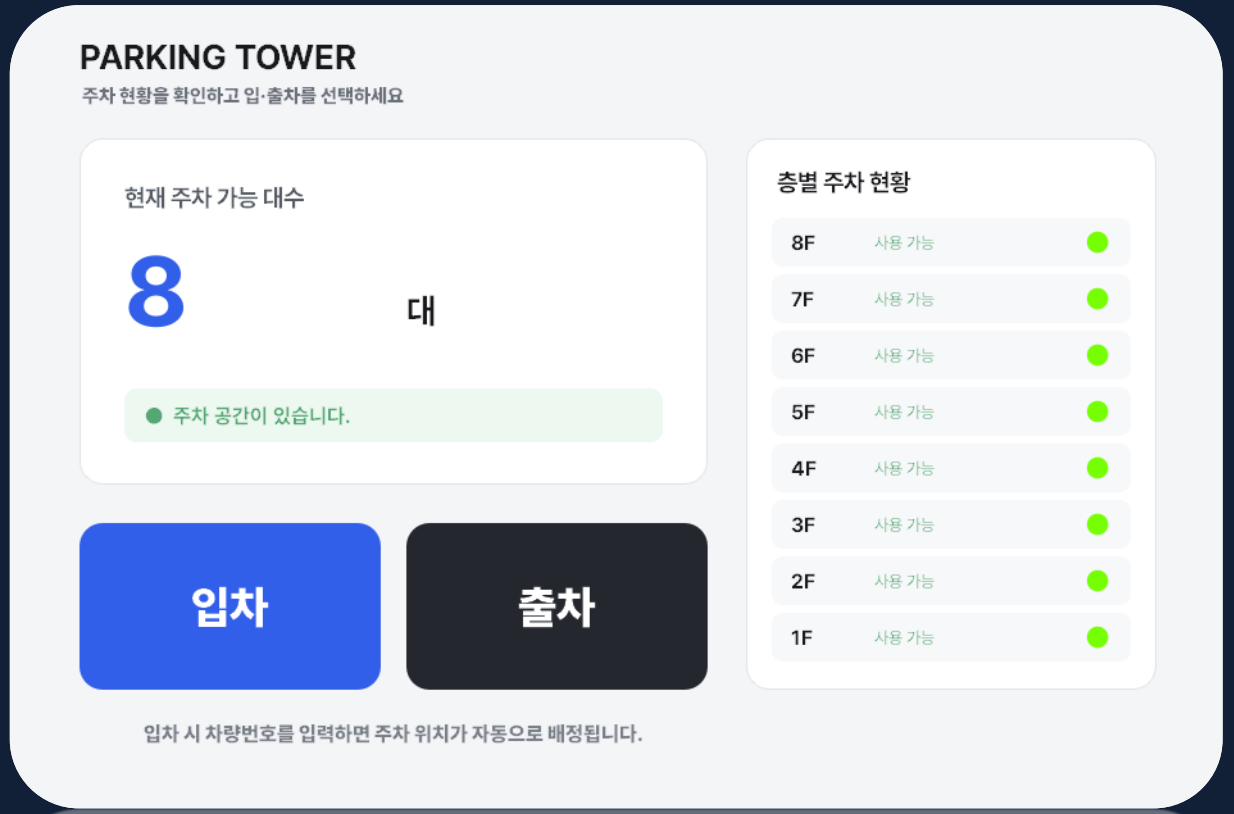
# QT 클래스 구조


ParkingTower [main]
ParkingTower.pro
Headers
car.h
mainwindow.h
parkingmanager.h
serialmanager.h
Sources
car.cpp
main.cpp
mainwindow.cpp
parkingmanager.cpp
serialmanager.cpp
Forms
mainwindow.ui
Resources
resources.qrc


MainWindow
-parkingManager : ParkingManager
-serialManager : SerialManager
+입차 처리()
+출차 처리()
+결제 처리()
+UI 상태 갱신()
ParkingManager
-cars : QVector<Car>
-slotStatus : quint8
+차량 등록()
+차량 검색()
+차량 삭제()
+주차 시간 계산()
+요금 계산()
+주차 상태 관리()
SerialManager
-uartSerial : QSerialPort
-rxBuffer : QByteArray
+Serial 연결()
+Serial 해제()
+데이터 송신()
+데이터 수신()
+STM32 신호 전달()
1
0..*
Car
-carNumber : QString
-floor : int
-entryTime : QDateTime
-exitTime : QDateTime
+차량번호 조회()
+주차층 조회()
+입차시간 조회()
+출차시간 설정()

# QT UI

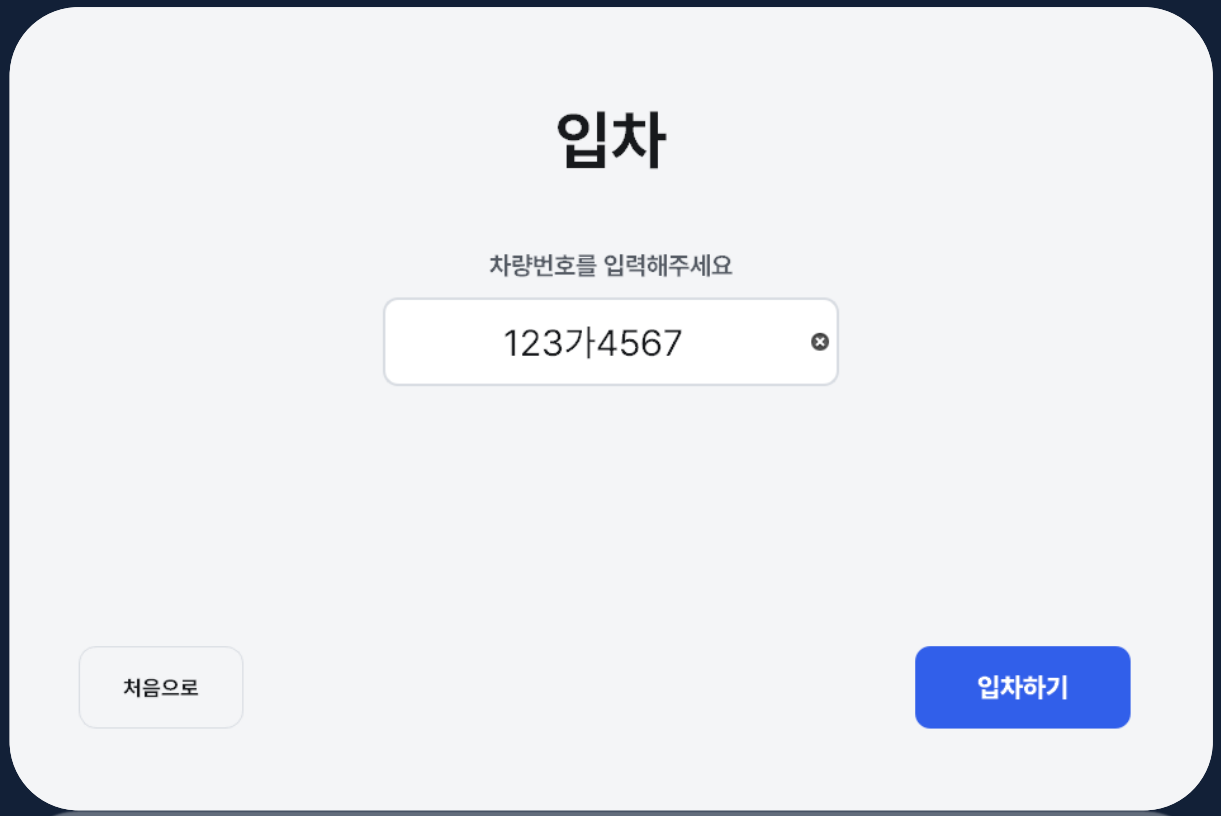
HOME | ENTRY | EXIT | PAYMENT | COMPLETE




- 홈 화면
- 현재 주차 가능 대수 표시
- 8개 슬롯 상태 - 사용 중 / 사용 가능
- 입차·출차 버튼 선택
  - 차량 감지시 입차 버튼 활성화

# QT UI

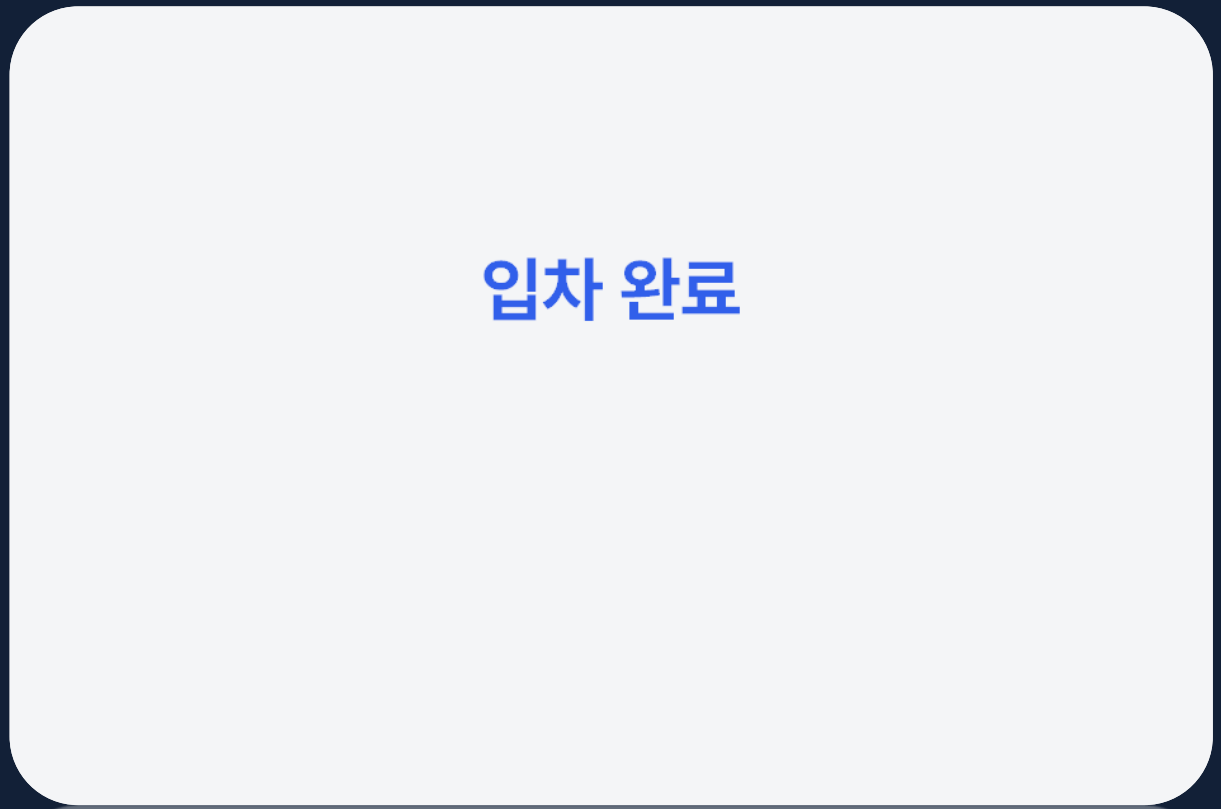
HOME | ENTRY | EXIT | PAYMENT | COMPLETE




- 차량 번호 입력: 2~3자리 + 한글 + 4자리 형식
- 정규식 검증
- 입차 요청

# QT UI

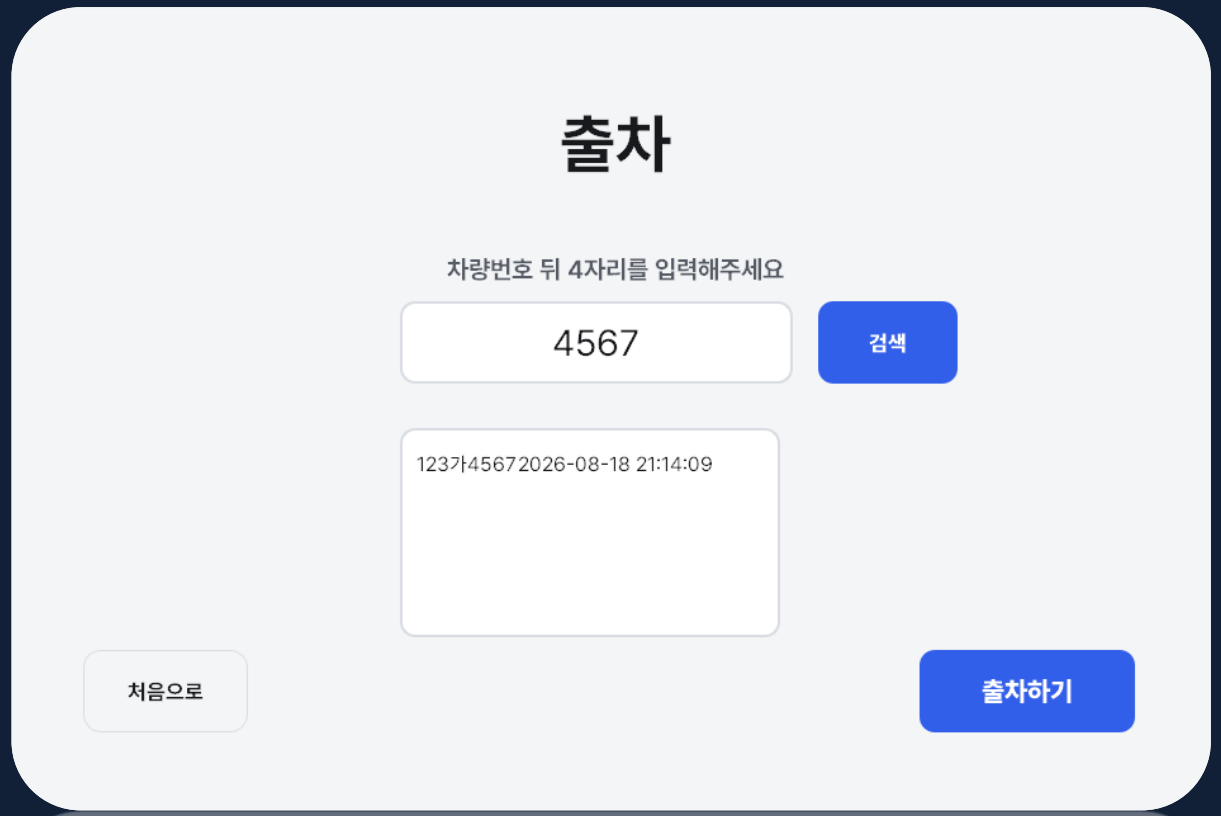
HOME | ENTRY | EXIT | PAYMENT | COMPLETE




- 차량 번호 입력: 2~3자리 + 한글 + 4자리 형식
- 정규식 검증
- 입차 요청

# QT UI

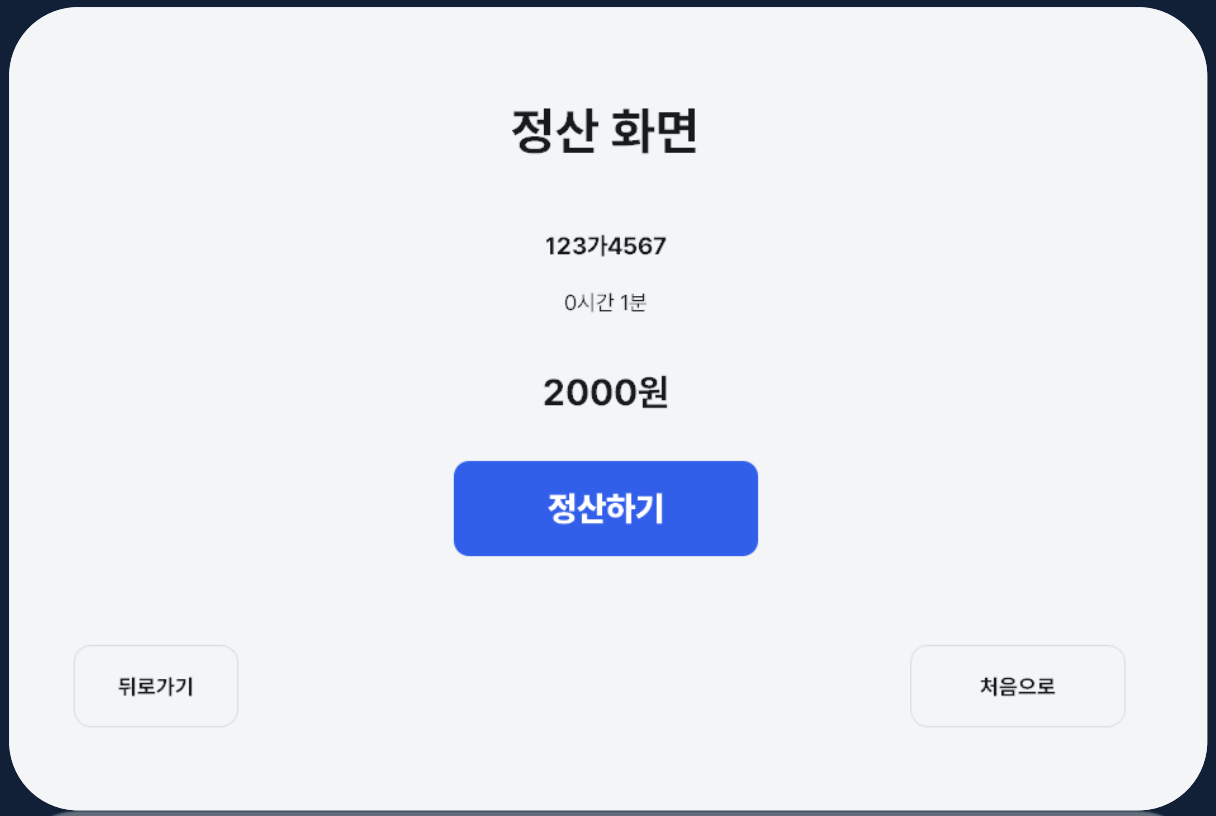
HOME | ENTRY | **EXIT** | PAYMENT | COMPLETE




- 출차 검색: 차량번호 마지막 4자리만 허용
- 차량 + 입차 시간 목록
- 출차 차량 선택

# QT UI

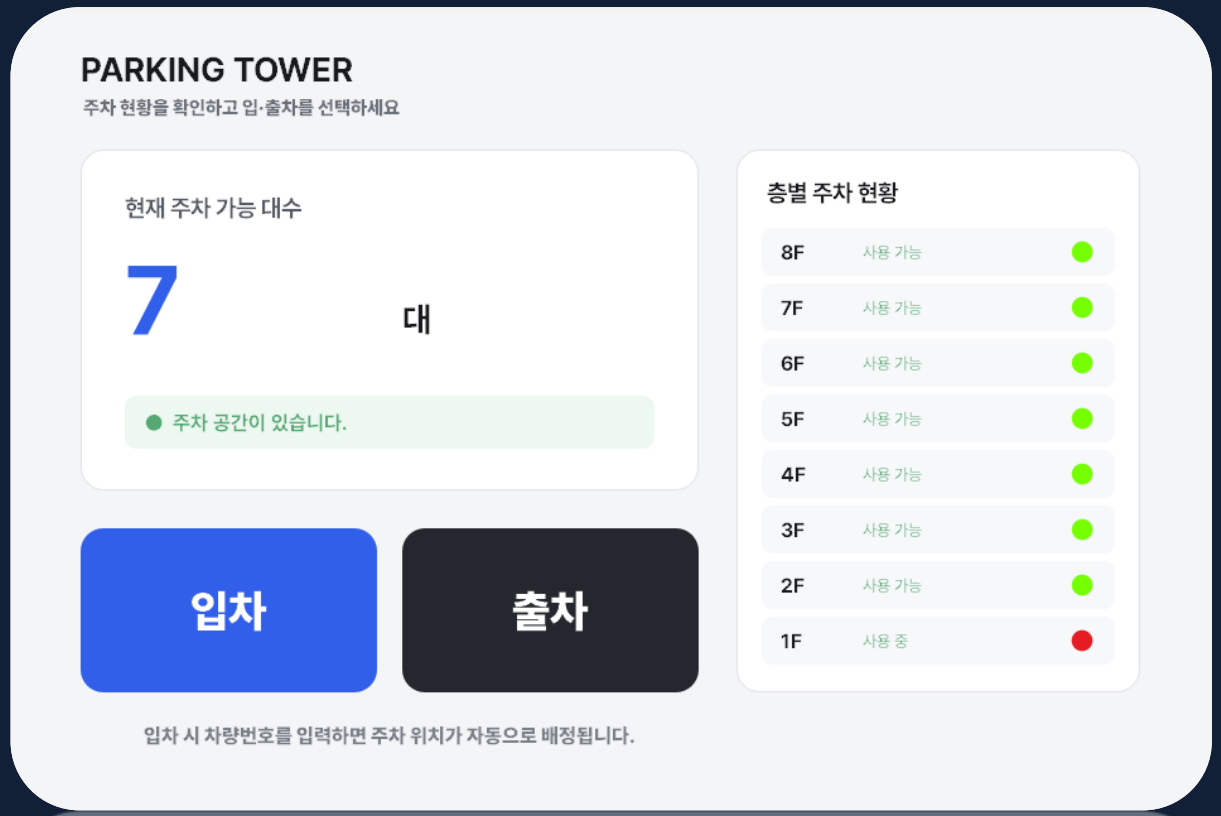
HOME | ENTRY | EXIT | **PAYMENT** | COMPLETE




- 차량 번호 표시
- 주차 시간 & 요금 표시
- 정산 요청

# QT UI

HOME | ENTRY | EXIT | PAYMENT | COMPLETE




- 입차/출차 완료 후 초기 화면
- 상태 재조회
- 슬롯 상태: green.png / red.png 이미지로 시각화

# UART 통신 프로토콜

### 통신 환경

115200bps / 8-N-1 / ASCII

수신 버퍼에 데이터를 누적하고
“\r\n“ 기준으로 명령을 판별

### STM32 ↔ QT 통신 프로토콜

**QT → STM32**

- **?** → 슬롯 상태 요청
- **I** → 특정 차량 입차 요청
- **O:3** → 3층 차량 출차 요청

**STM32 → QT**

- **R** → 시스템 Ready
- **D** → 차량 감지
- **S:00000101** → 슬롯 상태
- **P:3** → 3층 배정 완료
- **E** → 출차 완료

# UART 통신 주요 코드 (STM32)

```
/* ================================================================
          수신 명령 파싱 및 처리 로직 (프로토콜 규격 반영)
 * ================================================================ */

void UART_ParseCommand(char *cmd) {
    char tx_buf[64];
    //1. 입차 요청("I")
    if (strcmp(cmd, "I") == 0) {
            ........
            UART2_SendString(tx_buf);
            Servo_MoveBarrier();

        } else {
            UART2_SendString("E:3\r\n"); // 만차 시 에러 응답("E:3")
        }
    }
```

```
    //2. 정산 완료/출차 요청: "O:<floor>" (예: "O:3")
    else if (strncmp(cmd, "O:", 2) == 0) {
            ........
            UART2_SendString("E\r\n");
            Servo_MoveBarrier();
    }
    ........

    //3. 슬롯 점유 상태 조회: "?"
    else if (strcmp(cmd, "?") == 0) {
        char bitmask[9];
    ........

    }
    //4. 비정상 명령
    else {
        UART2_SendString("E:0\r\n");
    }
```

QT로부터 입력값을 받고, 입력값에 따라 정해진 정보 전달 및 동작 수행.

# UART 통신 기반 슬롯 상태 동기화 (QT)

```
void MainWindow::onSlotStatusReceived(quint8 slotStatus)
{
    quint8 storedStatus = parkingManager.getStoredSlotStatus();
    qDebug() << "Stored SLOT STATUS:"
             << QByteArray::number(storedStatus, 2).rightJustified(8, '0');

    qDebug() << "STM SLOT STATUS:"
             << QByteArray::number(slotStatus, 2).rightJustified(8, '0');
    if (storedStatus == slotStatus)
    {
        qDebug() << "SLOT STATUS MATCHED";
    }
    else
    {
        qDebug() << "SLOT STATUS MISMATCH";
    }
    parkingManager.setSlotStatus(slotStatus);
    updateHomeStatus();
    ui->stackedWidget->setCurrentWidget(ui->pageHome);
}
```

- 하드웨어로부터 주차 슬롯 상태를 수신했을 때 실행

# STM32 차량 감지 - 초음파 센서

```
if (distance <= 150) // 150mm 이하
{
    detect_count++;
    release_count = 0;

    if (detect_count >= 3) // 3회 연속 감지
    {
        detect_count = 3;
        car_state = CAR_DETECTED; // 차량 감지
    }
}
else if (distance >= 200) // 200mm 이상
{
    release_count++;
    detect_count = 0;

    if (release_count >= 3) // 3회 연속 해제
    {
        release_count = 3;
        car_state = CAR_NONE; // 차량 없음
    }
}
```

- 150mm 이내로 차량이 접근하면 감지
- 200mm 밖으로 차량이 벗어나면 차량 미존재로 인식

- 차량이 감지되면 “D\r\n” 송신

```
    if(previous_state != current_state)
{
    if(current_state == CAR_DETECTED)
    {
        UART2_SendString("D\r\n");
    }

    previous_state = current_state;
}
```

# STM32 차단기 제어 - 서보 모터

```
// 차단기 동작
void Servo_MoveBarrier(void){
    TIM2->CCR1 = 1800;  // 1.5ms (90도)
    TIM3_Delay(3000);

    TIM2->CCR1 = 800;  // 1.0ms (0도)
    TIM3_Delay(1000);
}
```

- QT로부터 수신한 명령어 파싱 결과에 따라 차단기 개방

# STM32 주차 슬롯 관리 및 자동 배정

**8개 주차 공간을 8-bit 상태값으로 관리**

8비트 상태를 수신 - **S:10011101**

bit 0 → 1층
bit 1 → 2층
bit 2 → 3층
bit 3 → 4층
bit 4 → 5층
bit 5 → 6층
bit 6 → 7층
bit 7 → 8층




- 0 = 사용 가능
- 1 = 사용 중
- 8개 모두 사용 → 만차

```
#define EMPTY_FLOOR   0b00000000
#define FULL_FLOOR    0b11111111
#define SLOT_ERROR    -1
```

# STM32 주차 슬롯 관리 및 자동 배정

### 최저층 우선 빈 슬롯 자동 탐색

**Case 1.**

1층 → 비어있음 → 1층 배정

**Case 2.**

1층 → 사용 중

2층 → 사용 중

3층 → 비어있음 → 3층 배정

```
// 가장낮은 층부터 비어있는 층반환 (1~8층 기준)
int Slot_FindLowestEmpty(){
    if (g_slot_occupancy_mask == FULL_FLOOR) {
        return SLOT_ERROR;
    }
    for (int i = 0; i < 8; i++) {
        if ((g_slot_occupancy_mask & (1U << i)) == 0U) {
            return i + 1;
        }
    }
    return SLOT_ERROR;
}
```

# QT 요금 계산 로직

## 기본 요금

기본 시간: 30분 | 기본 요금: 2,000원

## 추가 요금

기본 시간 초과 후 10분 단위로 500원 추가

## 상한

계산된 요금이 50,000원을 초과하면 최대 50,000원

```
int ParkingManager::calculateFee(qint64 parkingTime)
{
    if (parkingTime < 0)
        return -1;
    const int BasicTime = 30 * 60;
    const int BasicFee = 2000;
    const int ExtraTime = 10 * 60;
    const int ExtraFee = 500;
    const int MaxFee = 50000;

    int fee = BasicFee;
    if (parkingTime > BasicTime)
    {
        qint64 extraTime = parkingTime - BasicTime;

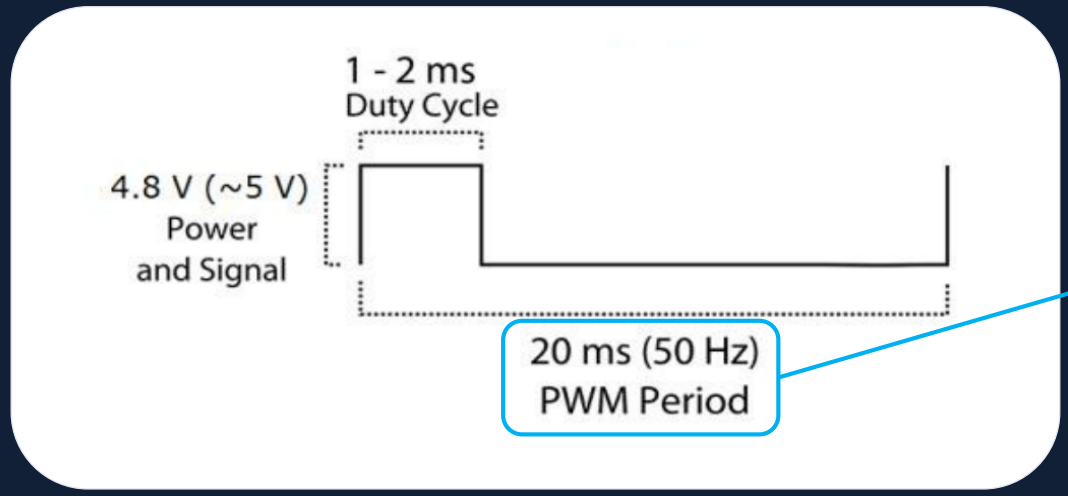
        qint64 extraCount = (extraTime + ExtraTime - 1) / ExtraTime;
        fee += extraCount * ExtraFee;
    }
    if (fee > MaxFee)
        fee = MaxFee;
    return fee;
}
```

# 향후 연구 과제

- 데이터 저장소 개선: CSV → SQLite 등 DB 기반 관리
- 차량 번호 자동 인식
- 차량번호 전체 검색, 중복 차량번호 검증, 비정상 입력 처리 강화
- UART 통신에 ACK / timeout / 재전송 / checksum 등을 추가해 통신 신뢰성 향상
- STM32와 Qt의 상태 동기화 실패 시 자동 복구 로직 추가
- 주차 이력 조회·검색·통계 화면 및 관리자 기능 확장
- 입구/출구 단말을 분리하고 다중 단말 통신 구조로 확장
- 실제 주차장 적용을 위한 장애 상황 및 동시 요청 테스트

# Trouble Shooting

- STM32 코드 병합 과정 중, 서보 모터가 동작하지 않는 문제 발생




SG90 서보 모터는 주기 20ms를 기준으로 동작

Position "0" (1.5 ms pulse) is middle, "90" (~2ms pulse) is middle,
is all the way to the right, "-90" (~1ms pulse) is all the way to the left.

움직이는 각도에 따라 듀티비를 설정.

# Trouble Shooting

- STM32 코드 병합 과정 중, 서보 모터가 동작하지 않는 문제 발생

```
void init_servo_motor(void){
    // 1. GPIOA 및 TIM2 클럭 인가
    Macro_Set_Bit(RCC->AHB1ENR, 0);     // GPIOA Clock Enable
    Macro_Set_Bit(RCC->APB1ENR, 0);     // TIM2 Clock Enable

    // 2. PA0 핀을 Alternate Function 모드로 설정
    Macro_Write_Block(GPIOA->MODER, 0x3, 2, 0);     // PA0 -> Mode 2 (AF)
    Macro_Write_Block(GPIOA->AFR[0], 0xF, 1, 0);    // PA0 -> AF1 (TIM2_CH1, 4비트 폭)

    // 3. 50Hz (20ms) 주기 설정 (96MHz 기준)
    TIM2->PSC = 15;
    TIM2->ARR = 19999;
    TIM2->CCR1 = 1000;                              // 초기 펄스폭: 1ms (0도)

    // 4. PWM 모드 1 설정 및 채널 1 출력 켜기
    // OC1M = 110 (PWM Mode 1, 비트 4~6), OC1PE = 1 (Preload Enable, 비트 3)
    Macro_Write_Block(TIM2->CCMR1, 0x7, 0x6, 4);
    Macro_Set_Bit(TIM2->CCMR1, 3);
```

STM32의 기본 주파수 : **16MHz**

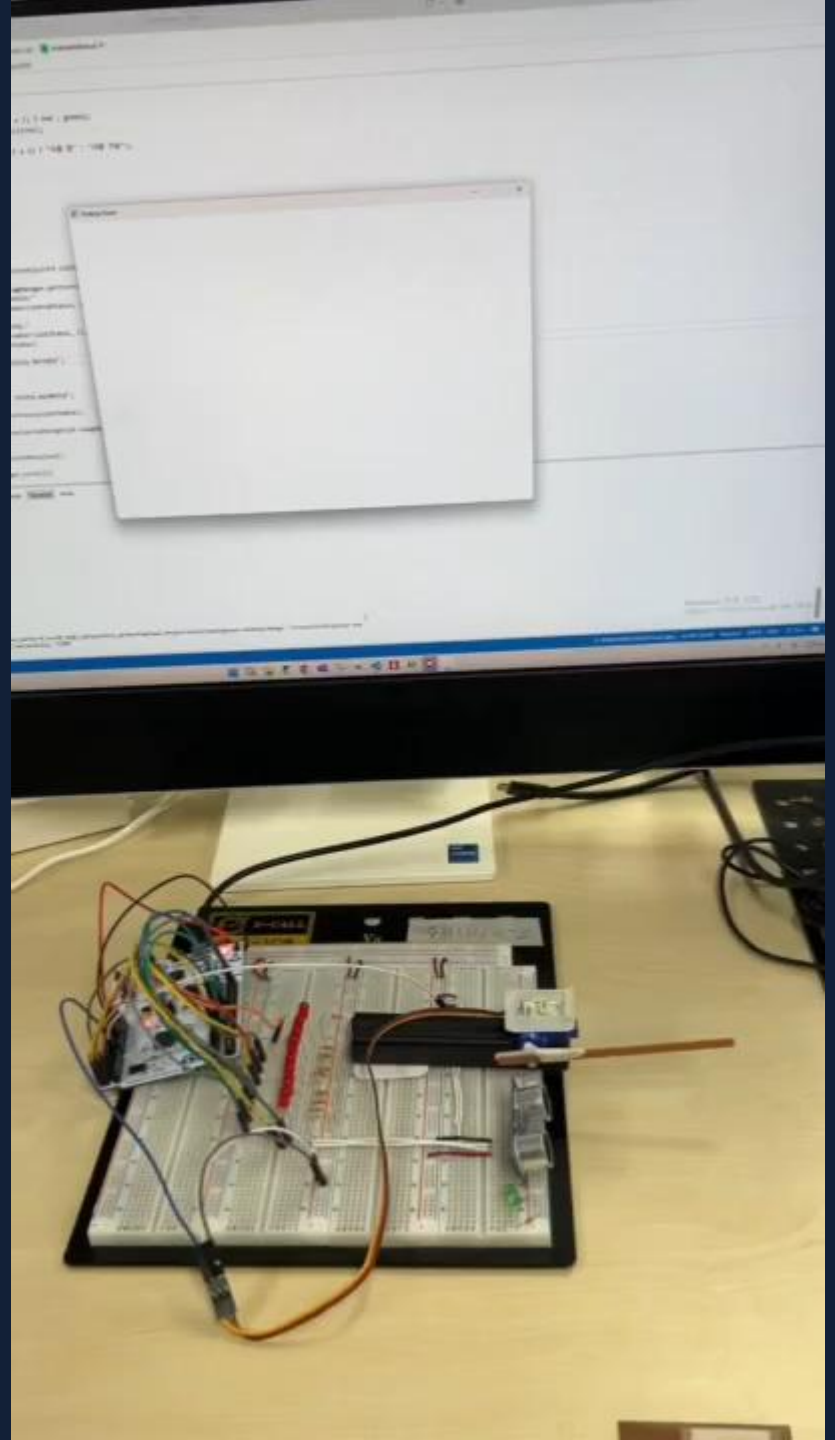
PLL (주파수 뻥튀기) 과정이 없다면,
16MHz를 기준으로 동작.

16MHz 기준 → **PSC = 15**
기존에는 96MHz 기준으로 **PSC = 95** 설정

PWM 파형이 6배 이상 길어지는 현상 발생

⇒ 당연히 서보 모터는 동작 X

# 시연 영상



# 프로젝트 수행 후기

1. 데이터 시트를 잘 참고하여 코드를 정확하게 작성해야 한다고 느낌.

2. QT와 MCU간의 데이터 송수신 과정을 전체적으로 파악할 수 있었음.

3. 문제 발생 시, 하드웨어와 소프트웨어 중 어느 곳이 문제인지 파악하는데 많은 시간을 할애했고,
   소프트웨어와 하드웨어 둘 다 깊은 지식이 필요하다고 느낌.

# 감사합니다